

FORMAL LANGUAGES AND COMPILERS

prof.s Luca Breveglieri and Angelo Morzenti

Exam of Fri 5 JULY 2019 - Part Theory

LAST + FIRST NAME:

(capital letters please)

MATRICOLA:

(or PERSON CODE)

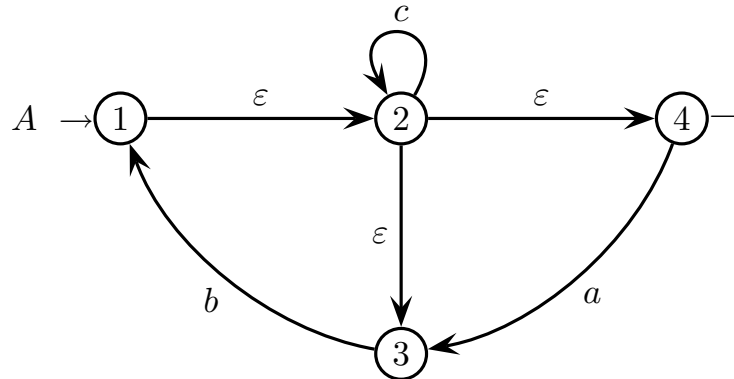
SIGNATURE:

INSTRUCTIONS - READ CAREFULLY:

- The exam is in written form and consists of two parts:
 1. Theory (80%): Syntax and Semantics of Languages
 - regular expressions and finite automata
 - free grammars and pushdown automata
 - syntax analysis and parsing methodologies
 - language translation and semantic analysis
 2. Lab (20%): Compiler Design by Flex and Bison
- To pass the exam, the candidate must succeed in both parts (theory and lab), in one call or more calls separately, but within one year (12 months) between the two parts.
- To pass part theory, the candidate must answer the mandatory (not optional) questions; notice that the full grade is achieved by answering the optional questions.
- The exam is open book: textbooks and personal notes are permitted.
- Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets and do not replace the existing ones.
- Time: part lab 60m - part theory 2h.15m

1 Regular Expressions and Finite Automata 20%

1. Consider the nondeterministic automaton A below, with spontaneous transitions, over the three-letter alphabet $\Sigma = \{a, b, c\}$:



Answer the following questions:

- (a) List all the strings of length from 0 (included) to 2 (included) that are accepted by automaton A , and separately list also all those (still of length from 0 to 2) that are rejected by A . Are the accepted strings ambiguous? Justify your answers.
- (b) Cut the spontaneous transitions of automaton A and obtain an equivalent automaton A' , possibly still nondeterministic. Check whether automaton A' is minimal, and if necessary minimize it.
- (c) Test the correctness of the minimal automaton A' : consider all the strings (accepted and rejected) listed at point (a) that do not contain letter c , and for each of such strings write either the accepting or the rejecting computation of A' .
- (d) By using the node elimination method, find a regular expression R that generates language $L(A)$. You can start from automaton A or A' , as best suits you. Test the correctness of R by means of the accepted strings (again only those that do not contain letter c) found at point (a).
- (e) Find the local sets $Ini(L(A))$, $Fin(L(A))$ and $Dig(L(A))$, and say if language $L(A)$ is local. Justify your answer.
- (f) (optional) Is the complement language $\overline{L(A)}$ local? Justify your answer.

2 Free Grammars and Pushdown Automata 20%

1. Consider the grammar G below, over a three-letter terminal alphabet $\{ a, b, c \}$ and a two-letter nonterminal alphabet $\{ X, S \}$ (axiom S):

$$G \left\{ \begin{array}{l} S \rightarrow a S X \mid \varepsilon \\ X \rightarrow b X \mid c X \mid \varepsilon \end{array} \right.$$

Answer the following questions:

- (a) Verify that grammar G is ambiguous, by drawing at least two syntax trees for the valid string below:

$a a b c$

- (b) For language $L(G)$, write a non-ambiguous grammar G' of the least general type.
- (c) (optional) Consider now the regular expression R below:

$$R = a^*(c b)^*$$

Is the intersection language $L(G) \cap L(R)$ regular ? Justify your answer.

2. Consider a variant of the language of the arithmetic expressions denoted in the *prefix form*, described as follows:

- An arithmetic expression can contain these operations and symbols:
 - n -ary addition, i.e., with $n \geq 2$ operands, denoted as $+$
 - binary multiplication, denoted as $*$
 - and also parentheses (and)
- Variables and numbers are schematized by terminal a , without expanding.
- Both addition and multiplication are denoted in the prefix form, e.g., $+ a a$ and $* a a$ for the binary case.
- Both addition and multiplication are associative operations, as usual.
- Binary addition, i.e., case $n = 2$, and multiplication never need parentheses, as usual with the prefix form.
- Ternary and in general n -ary addition, i.e., case $n \geq 3$, always needs parentheses, to identify the actual number n of operands, e.g., $(+ a a a)$ for case $n = 3$.

Examples (ten valid expressions):

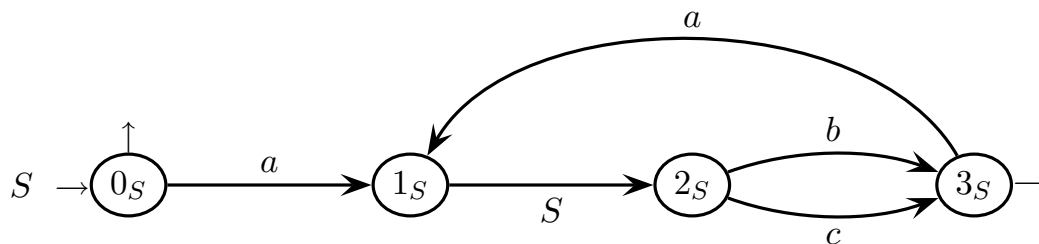
1. a
2. $+ a a$ // addition is binary, not parenthesized
3. $* a a$
4. $+ a + a a$ // both additions are binary, not parenthesized
5. $(+ a a a)$ // addition is ternary, parenthesized
6. $* + a a a$ // addition is binary, not parenthesized
7. $* (+ a a a) a$ // addition is ternary, parenthesized
8. $(+ + a a a a)$ // first addition is ternary, parenthesized
9. $+ (+ a a a) a$ // second addition is ternary, parenthesized
10. $(+ (+ a a a) a a)$ // both additions are ternary, parenthesized

Answer the following questions:

- (a) Write a grammar, of type *EBNF* and not ambiguous, that generates the sketched language of arithmetic expressions. The grammar generates one expression.
- (b) Test the correctness of your grammar by drawing the syntax trees of the sample arithmetic expressions n. 7, 8 and 9.
- (c) (optional) Argue (briefly but in a non-trivial and non-tautological manner) that your grammar is not ambiguous.

3 Syntax Analysis and Parsing Methodologies 20%

1. Consider the grammar G below, represented as a machine net, over a three-letter terminal alphabet $\Sigma = \{ a, b, c \}$ and a one-letter nonterminal alphabet (axiom S):



Answer the following questions (use the figures / tables / spaces on the next pages):

- (a) Draw the complete pilot of grammar G and show that grammar G is $ELR(1)$.
- (b) Write all the guide sets of the machine net of grammar G (on the call arcs and exit arrows) and show that grammar G is $ELL(1)$.
- (c) Write a simulation of the bottom-up analysis of the valid string below:

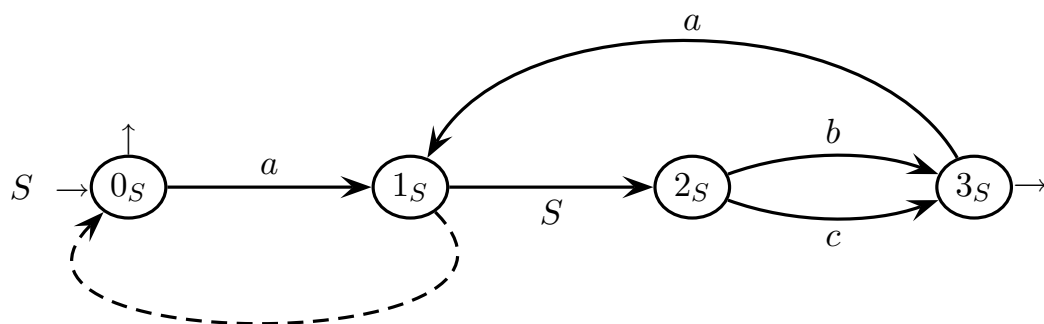
$$a a b a c b \in L(G)$$

Show at least the stack contents, the parsing actions and the resulting syntax tree.

- (d) (optional) Write the recursive descent procedure for nonterminal S , based on the ELL method.

please draw here the complete pilot – question (a)

please write here all the guide sets – question (b)



simulation table for the *ELR* analysis to be completed - question (c) - (the number of rows is not significant)

<i>stack of alternated macro-states and grammar symbols</i>	<i>input left to be scanned</i>	<i>move executed</i>
I_0	$a\ a\ b\ a\ c\ b\ \dashv$	initial configuration
$I_0\ a\ I_{\dots}$	$a\ b\ a\ c\ b\ \dashv$	terminal shift: $I_0 \xrightarrow{a} I_{\dots}$

draw here the piecewise tree construction

please write here the recursive descent procedure of nonterminal S – question (d)

4 Language Translation and Semantic Analysis 20%

1. Consider the source language L_s below, over the two-letter alphabet $\Sigma = \{ a, b \}$:

$$L_s = \{ a^h b^k \mid h, k \geq 0 \wedge (k \leq h \leq 2k \vee k > h) \}$$

A source grammar G_s that generates language L_s is (axiom S):

$$G_s \left\{ \begin{array}{l} S \rightarrow X \mid Y \\ X \rightarrow a a X b \mid a X b \mid \varepsilon \\ Y \rightarrow a Y b \mid Y b \mid b \end{array} \right.$$

Answer the following questions:

- (a) Write a syntax scheme G_τ (or a translation grammar) for the translation τ over the source language L_s defined as follows:

$$\tau(a^h b^k) = a^s b^s \quad s = \min(h, k)$$

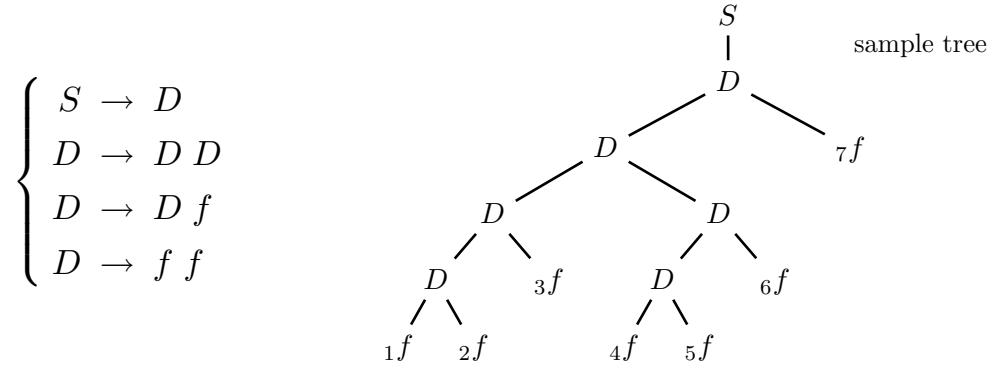
without changing the source grammar G_s .

Furthermore, draw the syntax translation trees of the two valid source strings:

$$a a b \quad a b b$$

- (b) Is scheme (or grammar) G_τ a deterministic translation ? Explain your answer.

2. The grammar below (axiom S), over a one-letter terminal alphabet $\{ f \}$, models the abstract syntax of the binary trees where all the internal nodes have two child nodes (simplified for the sake of brevity). A sample tree is reported on the right (a prefix subscript index is associated to every leaf node only for future reference):



This grammar just models an *abstract* syntax, which is unsuitable for syntax analysis.

We call *distance* between two tree nodes n_1 and n_2 , denoted as $dist(n_1, n_2)$, the length of the shortest path between them; for instance $dist(2f, 4f) = 6$ and $dist(2f, 7f) = 5$. The depth of any tree node is its distance from the root, hence in the sample tree the node depth ranges from 0 (for the root S) to 5 (for the leaf nodes $1f$, $2f$, $4f$ and $5f$).

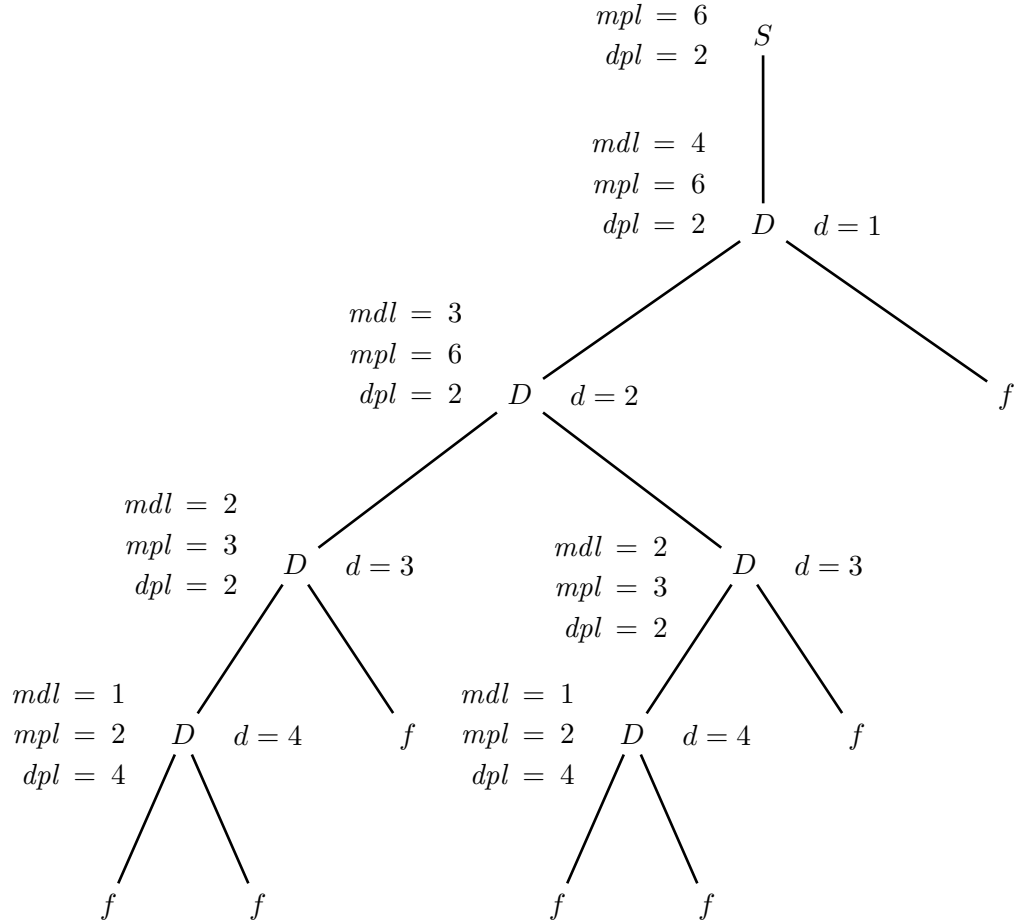
We intend to design an attribute grammar that supports the computation of the maximal distance between any two tree leaf nodes. In the above sample tree the maximal distance is 6, because for instance $dist(2f, 4f) = 6$.

The grammar must also allow for the computation of the depth of the deepest internal node that is a common ancestor of any two leaf nodes that have maximal distance. In the above example the depth is 2, because the deepest common ancestor of the leaf nodes $2f$ and $4f$ has depth 2 (it is the second node D from the root).

Consider the table below, which lists a set of attributes for the grammar:

<i>name</i>	<i>type</i>	<i>domain</i>	<i>symbol</i>	<i>meaning</i>
<i>d</i>	right	integer	D	d epth of the current node
<i>mdl</i>	left	integer	D	m aximal d istance, from the current node, of a l eaf node descendant of the current node
<i>mpl</i>	left	integer	D, S	m aximal distance between any p air of l eaf nodes both descendant of the current node
<i>dpl</i>	left	integer	D, S	d epth of the deepest internal node that is a common ancestor of a p air of l eaf nodes that are both descendant of the current node and have a maximal distance

The picture below shows the previous sample tree decorated with the attribute values:

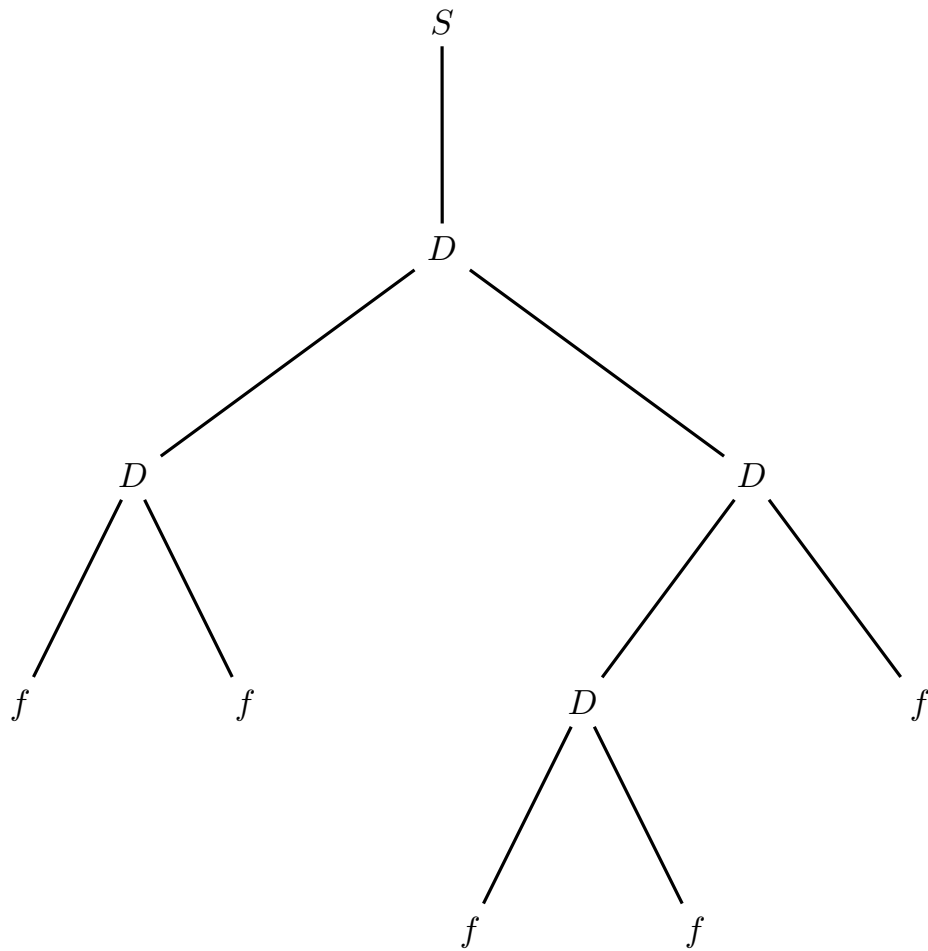


Answer the following questions (use the tables / drawings / spaces on the next pages):

- Write the semantic rules to compute attribute d .
- Write the semantic rules to compute attribute mdl . Decorate the next tree with the attribute value, according to the provided semantic rules.
- Write the semantic rules to compute attribute mpl . Decorate the next tree with the attribute value, according to the provided semantic rules.
- (optional) Write the semantic rules to compute attribute dpl . Decorate the next tree with the attribute value, according to the provided semantic rules.

When writing the semantic rules you can use functions like $\max(v_1, \dots, v_n)$, for any $n \geq 2$, to denote the maximum of a set of values $v_1, \dots, v_n$.

tree to be decorated for questions (b-c-d)



semantic rules for attribute d – question (a)

#	<i>syntax</i>	<i>semantics</i>
---	---------------	------------------

1:	$S_0 \rightarrow$	
----	-------------------	--

2:	$D_0 \rightarrow D_1 D_2$	
----	---------------------------	--

3:	$D_0 \rightarrow D_1 f$	
----	-------------------------	--

4:	$D_0 \rightarrow f f$	
----	-----------------------	--

semantic rules for attribute mdl – question (b)

#	<i>syntax</i>	<i>semantics</i>
---	---------------	------------------

1:	$S_0 \rightarrow$	
----	-------------------	--

2:	$D_0 \rightarrow D_1 D_2$	
----	---------------------------	--

3:	$D_0 \rightarrow D_1 f$	
----	-------------------------	--

4:	$D_0 \rightarrow f f$	
----	-----------------------	--

semantic rules for attribute *mpl* – question (c)

#	<i>syntax</i>	<i>semantics</i>
---	---------------	------------------

1:	$S_0 \rightarrow$	
----	-------------------	--

2:	$D_0 \rightarrow D_1 D_2$	
----	---------------------------	--

3:	$D_0 \rightarrow D_1 f$	
----	-------------------------	--

4:	$D_0 \rightarrow f f$	
----	-----------------------	--

semantic rules for attribute *dpl* – question (d)

#	<i>syntax</i>	<i>semantics</i>
---	---------------	------------------

1:	$S_0 \rightarrow$	
----	-------------------	--

2:	$D_0 \rightarrow D_1 D_2$	
----	---------------------------	--

3:	$D_0 \rightarrow D_1 f$	
----	-------------------------	--

4:	$D_0 \rightarrow f f$	
----	-----------------------	--
